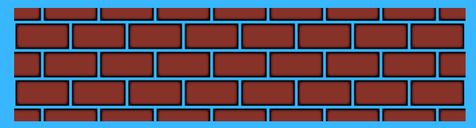
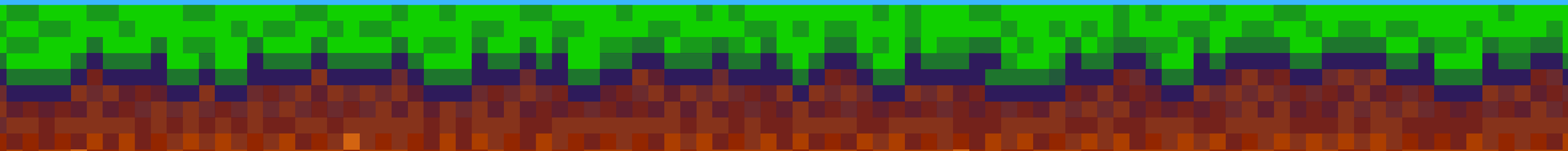


Trabalho Prático PDSII

Jogo de plataforma 2D

Eduardo Rodrigues
Felipe Alves
Lucas Gabriel
Mateus Paulo
Rafael Alves





CONTEXTO

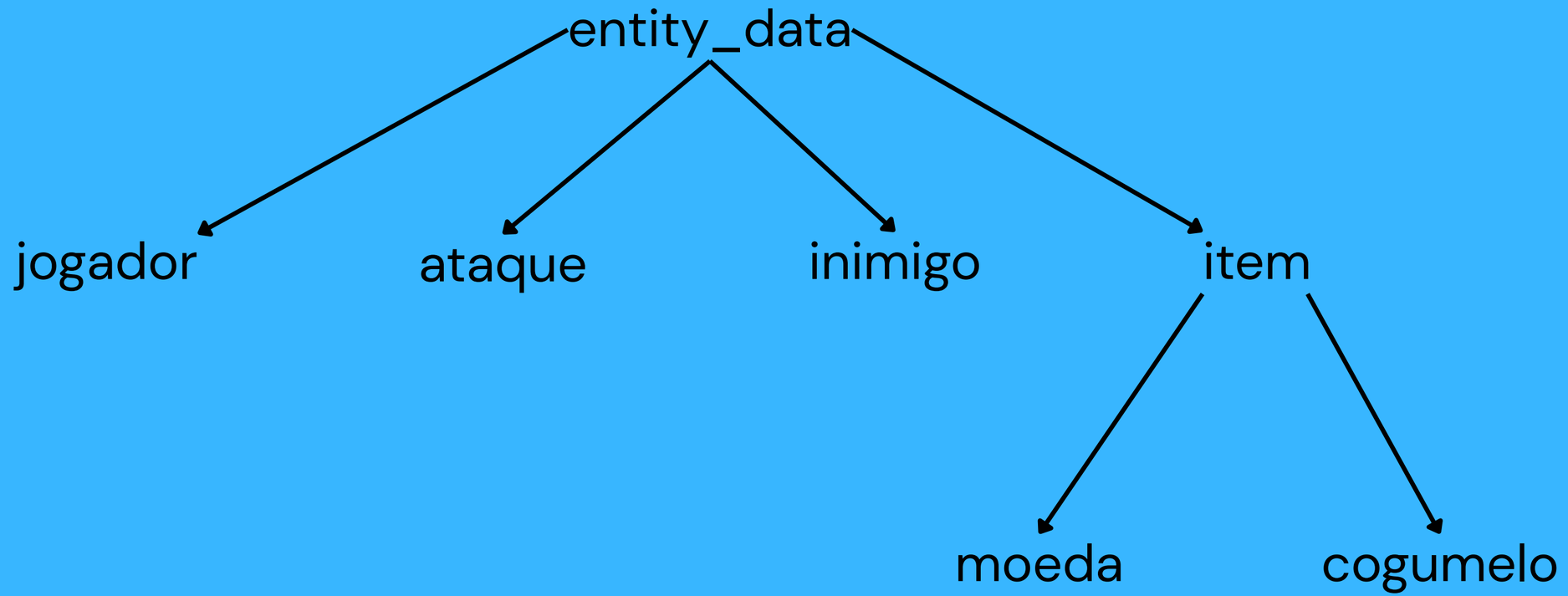
- Jogo de plataforma 2D
- Inspiração em Super Mario
- 3 fases rápidas com um chefão no final
- Controles simples e mecânica de tiro
- Uso de POO em C++



REPOSITÓRIO GITHUB

Pastas	Conteúdo
Build →	Armazena arquivos já compilados pelo makefile
Design →	Armazena os cartões CRC e UserStories junto ao arquivo que registra o recorde máximo e os mapas de fases do jogo
Include →	Armazena todos os hpps do jogo junto com as bibliotecas .h
Lib →	Armazena o código da biblioteca raylib para evitar dependências
Src →	Armazena todos os códigos cpps de include e a main
Tests →	Armazena todos os doctests para verificar erros do programa

DIAGRAMA DAS CLASSES



A seta aponta para o herdeiro da relação

ESTRATÉGIAS DE PROGRAMAÇÃO ADOTADAS

Raylib

- Biblioteca gráfica escolhida
- Simples, mas completa (equivalente ao allegro de PDS I)
- Dispensa uso de engines como Unity, Unreal ou Godot
- Biblioteca direcionada para jogos

unique_ptr

- Ponteiro inteligente de C++
- Não permite cópias, apenas mover
- Gerenciamento otimizado de memória

```
std::unique_ptr<Jogador> _jogador;
```

PROGRAMAÇÃO ORIENTADA A OBJETOS

Herança

- Entity_data é primitiva de todas as entidades do jogo
- Itens e inimigos são derivados de entity_data mas são primitivos de outras classes

Polimorfismo de funções

- Classe base Item possui a parte abstrata de funções implementadas nas suas derivadas

```
/**
 * @brief Aplica o efeito do item ao jogador que o coletou.
 * @param jogador Jogador que coletou o item.
 */
virtual void aplicarEfeito(Jogador& jogador) = 0;
```

```
void Cogumelo::aplicarEfeito(Jogador& jogador) {
    if (!confereVivo()) {
        return;
    }
    if (jogador.getEstado() == EstadoPoder::Pequeno) {
        jogador.crescer();
        jogador.adicionarPontos(_pontos);
    } else {    //Caso pegue outro cogumelo enquanto estiver sob
        jogador.adicionarPontos(_pontos * 2);
    }
    setAlive(false);
}
```

PROGRAMAÇÃO DEFENSIVA

- ErroArquivo
- ErroMapa
- ErroValorInvalido

→ Classes herdadas das exceções padrão de C++ para sinalizar qual tipo específico de erro ocorreu.

```
try {
    carregarFase(_totalFases);
} catch (const std::exception& e) {
    g_mensagemErro = e.what();
    TraceLog(LOG_ERROR, "Falha ao pular para o boss: %s", e.what());
}
```

Tratamento da exceção: o catch captura qualquer exceção derivada de std::exception, nosso caso. Desse jeito o erro é mostrado no menu, impresso no terminal e o jogo não fecha.

```
if (!std::isfinite(speed) || speed < 0.0f) {
    throw ErroValorInvalido("A velocidade deve ser um numero finito e nao negativo");
}
```

Exemplo de validação no construtor de codigo de entity_data para proteger contra velocidade negativa ou não finita.